

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО
ITMO University

ЛАБОРАТОРНАЯ РАБОТА №4

По дисциплине Промышленная маршрутизация и управление трафиком

Тема работы Эмуляция распределенной корпоративной сети связи, настройка iBGP, организация L3VPN, VPLS

Обучающийся Сакулин Иван Михайлович

Факультет Факультет прикладной информатики

Группа K3221

Направление подготовки 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа Программирование в инфокоммуникационных системах

Обучающийся

(дата)

(подпись)

Сакулин И.М.

(Ф.И.О.)

Руководитель

(дата)

(подпись)

Аминов Н.С.

(Ф.И.О.)

СОДЕРЖАНИЕ

Стр.

ВВЕДЕНИЕ	3
1 ХОД РАБОТЫ НАД VRF (L3VPN).....	4
1.1 Создание сети связи	4
1.2 Проверка пингов и локальной связности	10
2 ХОД РАБОТЫ НАД VPLS (L2VPN)	12
2.1 Проверка работы VPLS	14
ЗАКЛЮЧЕНИЕ	17

ВВЕДЕНИЕ

В отчёте представлен ход выполнения и результат работы.

Цель — изучить протоколы BGP, MPLS и правила организации L3VPN и VPLS.

Задачи:

- собрать IP/MPLS сеть связи;
- настроить IP адреса на интерфейсах;
- настроить OSPF;
- настроить MPLS;
- настроить iBGP;
- в первой версии настроить VRF (L3VPN);
- во второй части настроить VPLS (L2VPN);
- получить выкладки маршрутов, пингов, связности.

1 ХОД РАБОТЫ НАД VRF (L3VPN)

1.1 Создание сети связи

На рисунке 1 и 2 представлена конфигурация сети в containerlab. Созданы 6 маршрутизаторов (на базе RouterOS) и три компьютера (на базе Alpine Linux), сеть управления (mgmt) и связи между устройствами.

```
1  name: lab4a
2  mgmt:
3    network: mgmt
4    ipv4-subnet: 172.20.20.0/24
5
6  topology:
7    defaults:
8      kind: vr-ros
9      image: vrnetlab/mikrotik_routeros:6.47.9
10   nodes:
11     R01.HKI: # Хельсинки
12       kind: vr-ros
13       mgmt-ipv4: 172.20.20.11
14       startup-config: configs.a/R01.HKI.rsc
15     R01.SPB: # Санкт-Петербург
16       kind: vr-ros
17       mgmt-ipv4: 172.20.20.10
18       startup-config: configs.a/R01.SPB.rsc
19     PC1:
20       kind: linux
21       image: alpine:latest
22       cmd: sleep infinity
23       network-mode: none
24       exec:
25         - ip addr add 192.168.10.253/24 dev eth1
26         - ip route add default via 192.168.10.1
27     R02.LND: # Лондон
28       kind: vr-ros
29       mgmt-ipv4: 172.20.20.22
30       startup-config: configs.a/R02.LND.rsc
31     R02.NYD: # Нью-Йорк
32       kind: vr-ros
33       mgmt-ipv4: 172.20.20.20
34       startup-config: configs.a/R02.NYD.rsc
```

Рисунок 1 — Конфигурация сети в containerlab 1/2

```
1  PC2:
2    kind: linux
3    image: alpine:latest
4    cmd: sleep infinity
5    network-mode: none
6    exec:
7      - ip addr add 192.168.20.253/24 dev eth1
8      - ip route add default via 192.168.20.1
9  R03.LBN: # Лиссабон
10   kind: vr-ros
11   mgmt-ipv4: 172.20.20.33
12   startup-config: configs.a/R03.LBN.rsc
13  R03.SVL: # Савонlinna
14   kind: vr-ros
15   mgmt-ipv4: 172.20.20.30
16   startup-config: configs.a/R03.SVL.rsc
17  PC3:
18   kind: linux
19   image: alpine:latest
20   cmd: sleep infinity
21   network-mode: none
22   exec:
23     - ip addr add 192.168.30.253/24 dev eth1
24     - ip route add default via 192.168.30.1
25  links:
26    - endpoints: ["R01.HKI:eth1", "R02.LND:eth1"]
27    - endpoints: ["R01.HKI:eth2", "R03.LBN:eth1"]
28    - endpoints: ["R02.LND:eth2", "R03.LBN:eth2"]
29    - endpoints: ["R01.HKI:eth3", "R01.SPB:eth1"]
30    - endpoints: ["R02.LND:eth3", "R02.NYD:eth1"]
31    - endpoints: ["R03.LBN:eth3", "R03.SVL:eth1"]
32    - endpoints: ["R01.SPB:eth2", "PC1:eth1"]
33    - endpoints: ["R02.NYD:eth2", "PC2:eth1"]
34    - endpoints: ["R03.SVL:eth2", "PC3:eth1"]
```

Рисунок 2 — Конфигурация сети в containerlab 2/2

Для наглядности была составлена схемы логической топологии, представленная на рисунке 3.

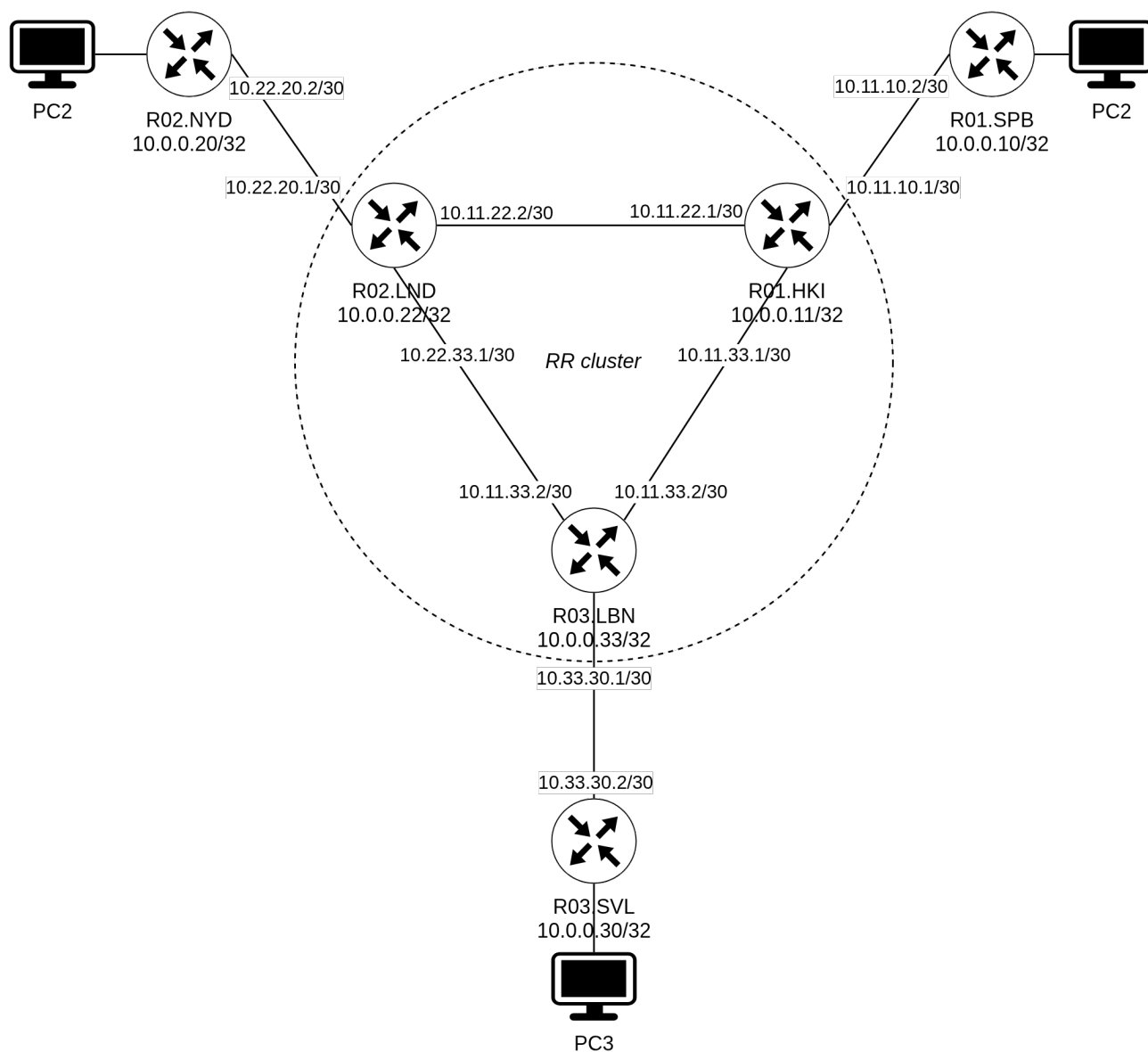
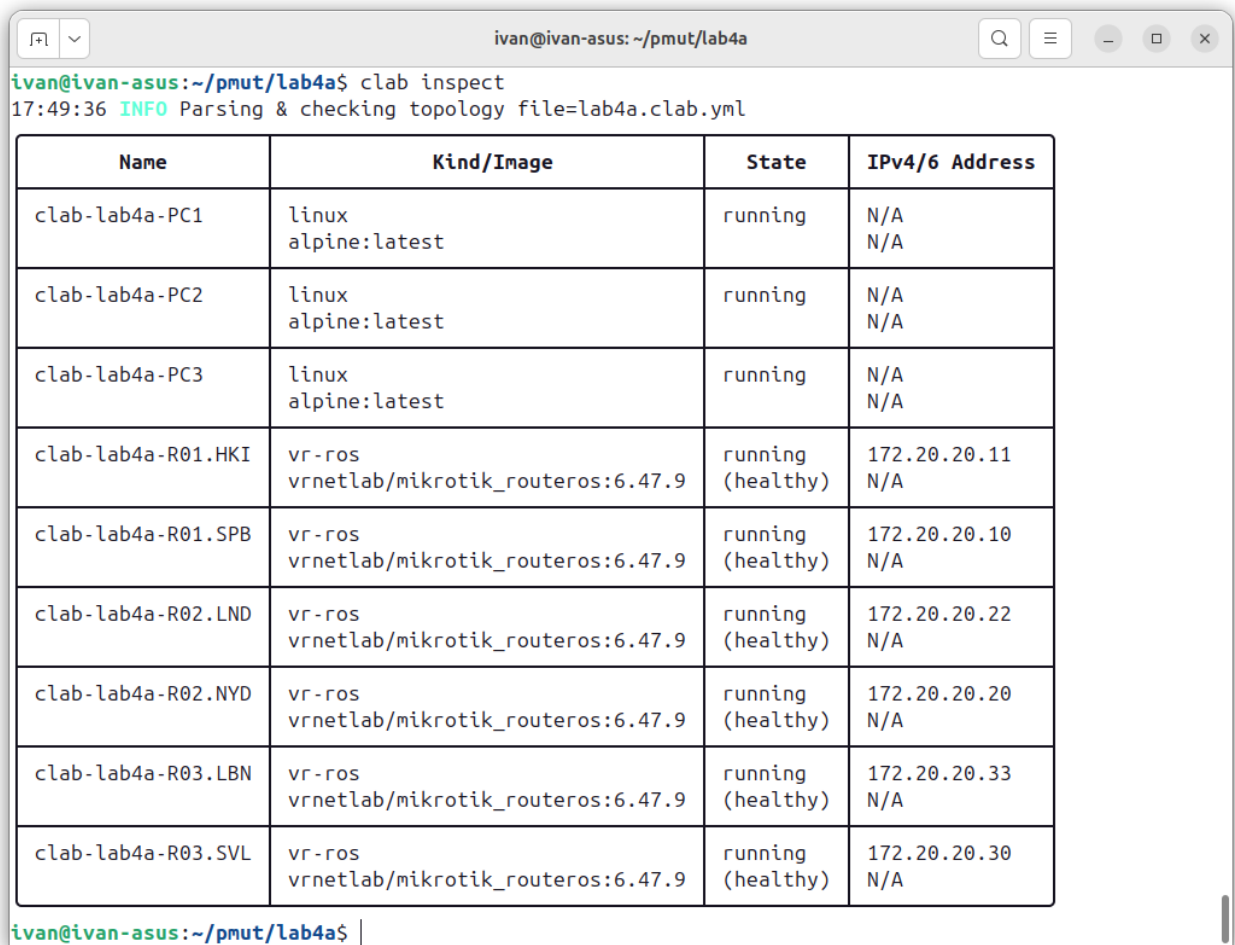


Рисунок 3 — Логическая схема сет (L3)

После написания конфигураций сеть была развёрнута с помощью containerlab (рисунок 4).



```
ivan@ivan-asus: ~/pmut/lab4a
ivan@ivan-asus:~/pmut/lab4a$ clab inspect
17:49:36 INFO Parsing & checking topology file=lab4a.clab.yml
```

Name	Kind/Image	State	IPv4/6 Address
clab-lab4a-PC1	linux alpine:latest	running	N/A N/A
clab-lab4a-PC2	linux alpine:latest	running	N/A N/A
clab-lab4a-PC3	linux alpine:latest	running	N/A N/A
clab-lab4a-R01.HKI	vr-ros vrnetlab/mikrotik_routeros:6.47.9	running (healthy)	172.20.20.11 N/A
clab-lab4a-R01.SPB	vr-ros vrnetlab/mikrotik_routeros:6.47.9	running (healthy)	172.20.20.10 N/A
clab-lab4a-R02.LND	vr-ros vrnetlab/mikrotik_routeros:6.47.9	running (healthy)	172.20.20.22 N/A
clab-lab4a-R02.NYD	vr-ros vrnetlab/mikrotik_routeros:6.47.9	running (healthy)	172.20.20.20 N/A
clab-lab4a-R03.LBN	vr-ros vrnetlab/mikrotik_routeros:6.47.9	running (healthy)	172.20.20.33 N/A
clab-lab4a-R03.SVL	vr-ros vrnetlab/mikrotik_routeros:6.47.9	running (healthy)	172.20.20.30 N/A

```
ivan@ivan-asus:~/pmut/lab4a$
```

Рисунок 4 — Развёртывание сети в containerlab

Ниже приведены фрагменты конфигураций маршрутизаторов. На всех роутерах заданы имена, пароль, адресация на интерфейсах, loopback интерфейсы (10.0.0.x/32), настроен OSPF в area backbone, включён MPLS LDP на всех физических интерфейсах между роутерами.

На примере R01.HKI (маршрутизатор кластера) показана настройка iBGP с поддержкой VPNv4 и назначение route reflectors для клиентов (рисунок 5).

```
1 /system identity set name=R01.HKI
2 /user set [find name=admin] password=cfrekby
3
4 # IP
5 /interface bridge add name=lo0
6 /ip address
7 add address=10.0.0.11/32 interface=lo0
8 add address=10.11.22.1/30 interface=ether2 comment="to R02.LND"
9 add address=10.11.33.1/30 interface=ether3 comment="to R03.LBN"
10 add address=10.11.10.1/30 interface=ether4 comment="to R01.SPB"
11
12 # OSPF
13 /routing ospf instance set default router-id=10.0.0.11
14 /routing ospf network
15 add network=10.0.0.11/32 area=backbone
16 add network=10.11.22.0/30 area=backbone
17 add network=10.11.33.0/30 area=backbone
18 add network=10.11.10.0/30 area=backbone
19
20 # MPLS
21 /mpls ldp set enabled=yes transport-address=10.0.0.11 lsr-id=10.0.0.11
22 /mpls ldp interface
23 add interface=ether2
24 add interface=ether3
25 add interface=ether4
26
27 # iBGP RR
28 /routing bgp instance set default as=65000 router-id=10.0.0.11 redistribute-connected=yes
29
30 /routing bgp peer
31 # Cluster RR
32 add name=to_LND remote-address=10.0.0.22 remote-as=65000 update-source=lo0
33 ↪ address-families=vpn4
34 add name=to_LBN remote-address=10.0.0.33 remote-as=65000 update-source=lo0
35 ↪ address-families=vpn4
36
37 # Client (SPB)
38 add name=to_SPB remote-address=10.0.0.10 remote-as=65000 update-source=lo0 route-reflect=yes
39 ↪ address-families=vpn4
```

Рисунок 5 — Настройки маршрутизатора R01.HKI (RR)

На граничных маршрутизаторах (PE) настроены VRF, интерфейс подключения к клиенту и BGP для обмена маршрутами VPNv4. Пример для R01.SPB представлен на рисунке 6.

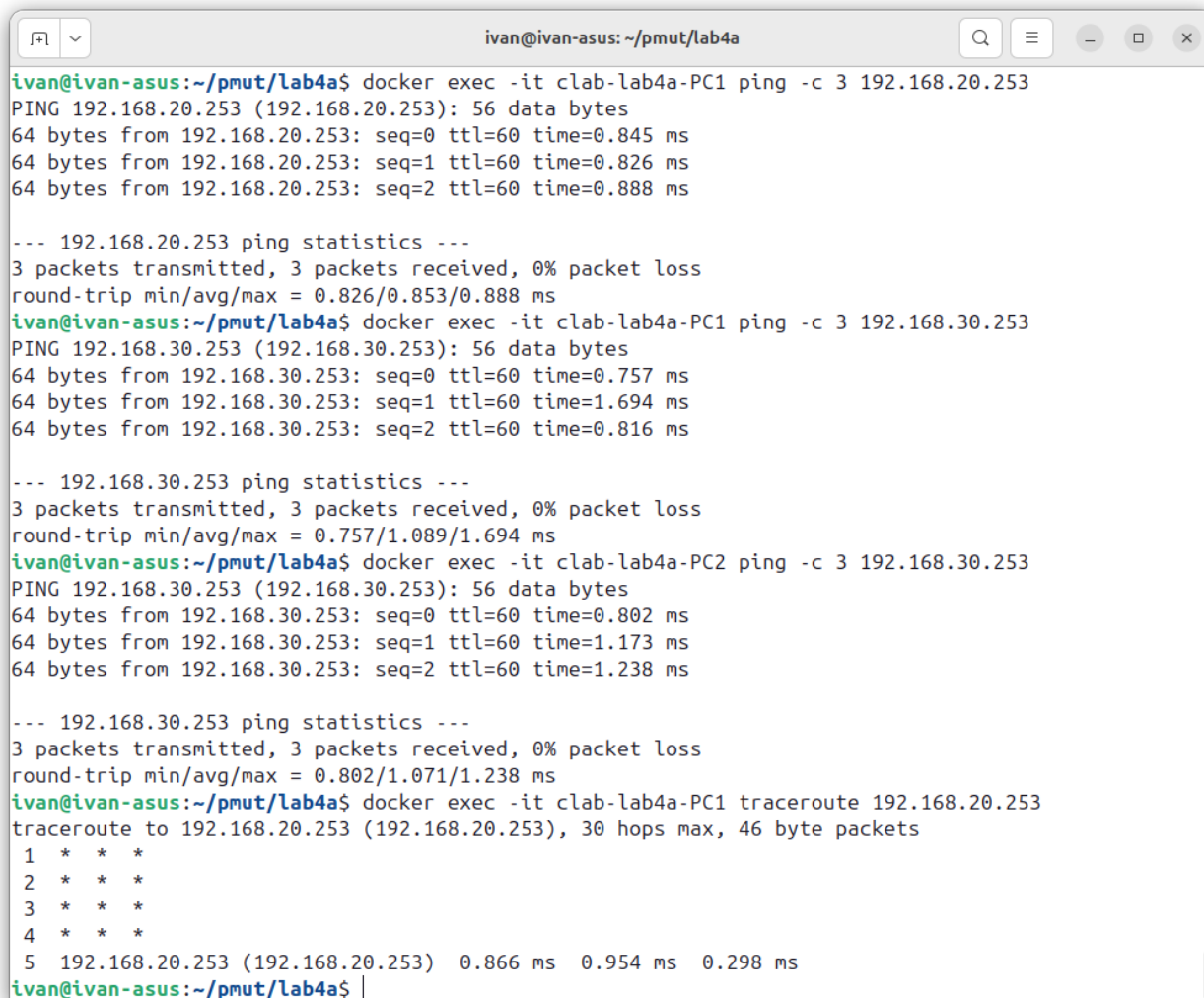
```
1 /system identity set name=R01.SPB
2 /user set [find name=admin] password=cfrekby
3
4 # IP
5 /interface bridge add name=lo0
6 /ip address
7 add address=10.0.0.10/32 interface=lo0
8 add address=10.11.10.2/30 interface=ether2 comment="to R01.HKI"
9
10 # OSPF
11 /routing ospf instance set default router-id=10.0.0.10
12 /routing ospf network
13 add network=10.0.0.10/32 area=backbone
14 add network=10.11.10.0/30 area=backbone
15
16 # MPLS
17 /mpls ldp set enabled=yes transport-address=10.0.0.10 lsr-id=10.0.0.10
18 /mpls ldp interface add interface=ether2
19
20 # iBGP
21 /routing bgp instance set default as=65000 router-id=10.0.0.10
22 /routing bgp peer add name=to_RR_HKI remote-address=10.0.0.11 remote-as=65000
   ↪ update-source=lo0 address-families=vpn4
23
24 # VRF
25 /ip route vrf add routing-mark=VRF_DEVOPS interfaces=ether3 route-distinguisher=65000:1
   ↪ import-route-targets=65000:1 export-route-targets=65000:1
26 /ip address add address=192.168.10.1/24 interface=ether3
27
28 # BGP VRF
29 /routing bgp instance vrf add routing-mark=VRF_DEVOPS instance=default
   ↪ redistribute-connected=yes
```

Рисунок 6 — Настройки маршрутизатора R01.SPB (PE)

Аналогичные настройки выполнены на R02.NYD и R03.SVL с соответствующими адресами подсетей клиентов.

1.2 Проверка пингов и локальной связности

После развёртывания сети была проверена связность между клиентскими устройствами. На рисунке 7 показаны результаты ping и traceroute от PC1 (192.168.10.253) до PC2 (192.168.20.253) и PC3 (192.168.30.253).

A screenshot of a terminal window titled 'ivan@ivan-asus: ~/pmut/lab4a'. The terminal shows a series of commands and their outputs. First, a ping command is run from a container named 'clab-lab4a-PC1' to the IP 192.168.20.253. The output shows three successful pings with times around 0.8 ms. Then, another ping is run to 192.168.30.253, also successful with times around 0.8 to 1.7 ms. Next, a ping is run from 'clab-lab4a-PC2' to 192.168.30.253, which is also successful. Finally, a traceroute is performed from 'clab-lab4a-PC1' to 192.168.20.253. The traceroute shows four hops, all marked with asterisks, indicating successful connections. The final hop to 192.168.20.253 shows times of 0.866 ms, 0.954 ms, and 0.298 ms.

```
ivan@ivan-asus:~/pmut/lab4a$ docker exec -it clab-lab4a-PC1 ping -c 3 192.168.20.253
PING 192.168.20.253 (192.168.20.253): 56 data bytes
64 bytes from 192.168.20.253: seq=0 ttl=60 time=0.845 ms
64 bytes from 192.168.20.253: seq=1 ttl=60 time=0.826 ms
64 bytes from 192.168.20.253: seq=2 ttl=60 time=0.888 ms

--- 192.168.20.253 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.826/0.853/0.888 ms
ivan@ivan-asus:~/pmut/lab4a$ docker exec -it clab-lab4a-PC1 ping -c 3 192.168.30.253
PING 192.168.30.253 (192.168.30.253): 56 data bytes
64 bytes from 192.168.30.253: seq=0 ttl=60 time=0.757 ms
64 bytes from 192.168.30.253: seq=1 ttl=60 time=1.694 ms
64 bytes from 192.168.30.253: seq=2 ttl=60 time=0.816 ms

--- 192.168.30.253 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.757/1.089/1.694 ms
ivan@ivan-asus:~/pmut/lab4a$ docker exec -it clab-lab4a-PC2 ping -c 3 192.168.30.253
PING 192.168.30.253 (192.168.30.253): 56 data bytes
64 bytes from 192.168.30.253: seq=0 ttl=60 time=0.802 ms
64 bytes from 192.168.30.253: seq=1 ttl=60 time=1.173 ms
64 bytes from 192.168.30.253: seq=2 ttl=60 time=1.238 ms

--- 192.168.30.253 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.802/1.071/1.238 ms
ivan@ivan-asus:~/pmut/lab4a$ docker exec -it clab-lab4a-PC1 traceroute 192.168.20.253
traceroute to 192.168.20.253 (192.168.20.253), 30 hops max, 46 byte packets
 1 * * *
 2 * * *
 3 * * *
 4 * * *
 5 192.168.20.253 (192.168.20.253) 0.866 ms 0.954 ms 0.298 ms
ivan@ivan-asus:~/pmut/lab4a$
```

Рисунок 7 — Проверка связности между клиентами L3VPN

Также была проверена корректность маршрутизации в VRF на PE-роутерах. На рисунке 8 приведён вывод маршрутов VRF для R01.SPB, R02.NYD, R03.SVL.

```
ivan@ivan-asus: ~/pmut/lab4a
ivan@ivan-asus:~/pmut/lab4a$ sshpass -p cfrekby ssh admin@172.20.20.11 "/mpls ldp neighbor print"
Flags: X - disabled, D - dynamic, O - operational, T - sending-targeted-hello,
V - vpls
#      TRANSPORT      LOCAL-TRANSPORT PEER      SEN
0 DO   10.0.0.22       10.0.0.11       10.0.0.22:0      no
1 DO   10.0.0.33       10.0.0.11       10.0.0.33:0      no
2 DO   10.0.0.10       10.0.0.11       10.0.0.10:0      no

ivan@ivan-asus:~/pmut/lab4a$ sshpass -p cfrekby ssh admin@172.20.20.11 "/routing bgp peer print"
Flags: X - disabled, E - established
#      INSTANCE      REMOTE-ADDRESS      REMOTE-AS
0 E default 10.0.0.22           65000
1 E default 10.0.0.33           65000
2 E default 10.0.0.10           65000

ivan@ivan-asus:~/pmut/lab4a$ sshpass -p cfrekby ssh admin@172.20.20.10 "/ip route print where routing-mark=VRF_DEVOPS"
Flags: X - disabled, A - active, D - dynamic,
C - connect, S - static, r - rip, b - bgp, o - ospf, m - mme,
B - blackhole, U - unreachable, P - prohibit
#      DST-ADDRESS      PREF-SRC      GATEWAY      DISTANCE
0 ADC 192.168.10.0/24    192.168.10.1  ether3       0
1 ADb 192.168.20.0/24    10.0.0.20     200
2 ADb 192.168.30.0/24    10.0.0.30     200

ivan@ivan-asus:~/pmut/lab4a$ sshpass -p cfrekby ssh admin@172.20.20.20 "/ip route print where routing-mark=VRF_DEVOPS"
Flags: X - disabled, A - active, D - dynamic,
C - connect, S - static, r - rip, b - bgp, o - ospf, m - mme,
B - blackhole, U - unreachable, P - prohibit
#      DST-ADDRESS      PREF-SRC      GATEWAY      DISTANCE
0 ADb 192.168.10.0/24    10.0.0.10     200
1 ADC 192.168.20.0/24    192.168.20.1  ether3       0
2 ADb 192.168.30.0/24    10.0.0.30     200

ivan@ivan-asus:~/pmut/lab4a$ sshpass -p cfrekby ssh admin@172.20.20.30 "/ip route print where routing-mark=VRF_DEVOPS"
Flags: X - disabled, A - active, D - dynamic,
C - connect, S - static, r - rip, b - bgp, o - ospf, m - mme,
B - blackhole, U - unreachable, P - prohibit
#      DST-ADDRESS      PREF-SRC      GATEWAY      DISTANCE
0 ADb 192.168.10.0/24    10.0.0.10     200
1 ADb 192.168.20.0/24    10.0.0.20     200
2 ADC 192.168.30.0/24    192.168.30.1  ether3       0

ivan@ivan-asus:~/pmut/lab4a$
```

Рисунок 8 — Проверка маршрутов VRF на PE

Для оценки работы MPLS и iBGP выполнены команды `mpls ldp neighbor print` и `routing bgp peer print` на маршрутизаторе класера. Все соседства установлены, LDP сессии активны, BGP пиры находятся в состоянии Established.

2 ХОД РАБОТЫ НАД VPLS (L2VPN)

Для второй части лабораторной работы конфигурация containerlab была изменена: клиентские компьютеры получили адреса из одной общей подсети 192.168.100.0/24, а на граничных маршрутизаторах вместо VRF был настроен VPLS (L2VPN) с использованием BGP-сигнализации.

Файл конфигурации второй части содержит те же узлы, но с изменёнными адресами PC и параметрами запуска (рисунок 9).

```
1 name: lab4b
2 # ... mgmt сеть аналогична
3 topology:
4   nodes:
5     R01.HKI:
6       startup-config: configs.b/R01.HKI.rsc
7     R01.SPB:
8       startup-config: configs.b/R01.SPB.rsc
9     PC1:
10      exec:
11        - ip addr add 192.168.100.1/24 dev eth1
12     R02.LND:
13       startup-config: configs.b/R02.LND.rsc
14     R02.NYD:
15       startup-config: configs.b/R02.NYD.rsc
16     PC2:
17      exec:
18        - ip addr add 192.168.100.2/24 dev eth1
19     R03.LBN:
20       startup-config: configs.b/R03.LBN.rsc
21     R03.SVL:
22       startup-config: configs.b/R03.SVL.rsc
23     PC3:
24      exec:
25        - ip addr add 192.168.100.3/24 dev eth1
26 links: # без изменений
```

Рисунок 9 — Конфигурация VPLS в containerlab

Конфигурации маршрутизаторов претерпели изменения: на RR добавлена address-family l2vpn, на PE вместо VRF настраивается VPLS. Пример для R01.HKI (RR) представлен на рисунке 10.

```
1 /system identity set name=R01.HKI
2 # ... IP, OSPF, MPLS без изменений
3 /routing bgp instance set default as=65000 router-id=10.0.0.11 redistribute-connected=yes
  ↪ client-to-client-reflection=yes
4
5 /routing bgp peer
6 add name=to_LND remote-address=10.0.0.22 remote-as=65000 update-source=lo0
  ↪ address-families=vpn4,l2vpn
7 add name=to_LBN remote-address=10.0.0.33 remote-as=65000 update-source=lo0
  ↪ address-families=vpn4,l2vpn
8 add name=to_SPB remote-address=10.0.0.10 remote-as=65000 update-source=lo0 route-reflect=yes
  ↪ address-families=vpn4,l2vpn
```

Рисунок 10 — Настройки R01.HKI для поддержки VPLS

На PE (R01.SPB) создаётся bridge, в него добавляется интерфейс к клиенту, и настраивается интерфейс VPLS с использованием BGP (рисунок 11).

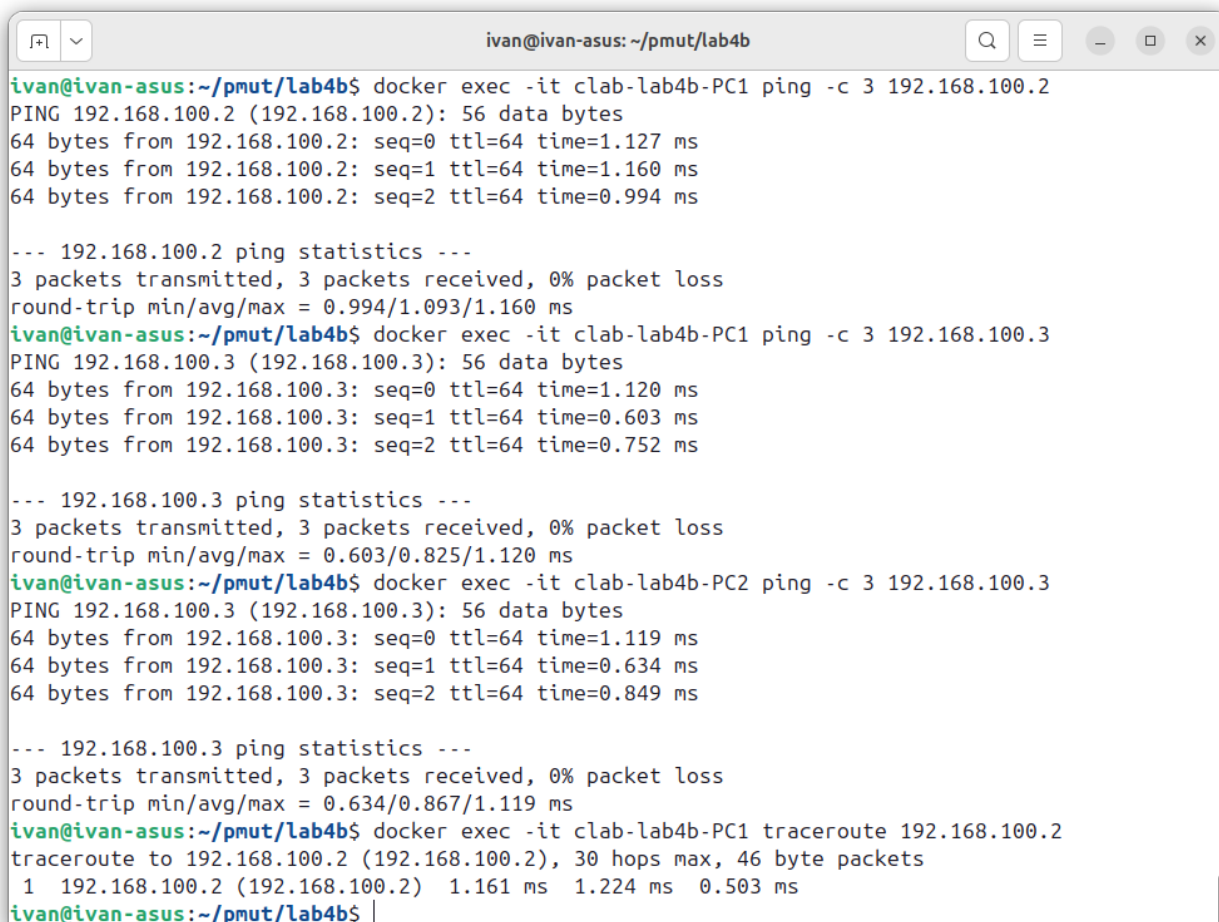
```
1 /system identity set name=R01.SPB
2 # ... IP, OSPF, MPLS
3 /routing bgp instance set default as=65000 router-id=10.0.0.10
4 /routing bgp peer add name=to_RR_HKI remote-address=10.0.0.11 remote-as=65000
  ↪ update-source=lo0 address-families=vpn4,l2vpn
5
6 # VPLS BGP Setup
7 /interface bridge add name=br-vpls
8 /interface bridge port add bridge=br-vpls interface=ether3
9
10 /interface vpls bgp-vpls add name=vpls_bgp bridge=br-vpls site-id=10
  ↪ route-distinguisher=65000:100 export-route-targets=65000:100
  ↪ import-route-targets=65000:100
```

Рисунок 11 — Настройки R01.SPB (VPLS PE)

Аналогично настроены R02.NYD (site-id=20) и R03.SVL (site-id=30) с единым route-target 65000:100.

2.1 Проверка работы VPLS

После запуска сети проверена связность между клиентами, которые теперь находятся в одной широковещательной домене. Результаты ping и traceroute показаны на рисунке 12.

A screenshot of a terminal window titled 'ivan@ivan-asus: ~/pmut/lab4b'. The terminal shows a series of commands and their outputs. The user runs 'docker exec -it clab-lab4b-PC1 ping -c 3 192.168.100.2', which shows successful ping results for 192.168.100.2. Then, the user runs 'docker exec -it clab-lab4b-PC1 ping -c 3 192.168.100.3', showing successful ping results for 192.168.100.3. Finally, the user runs 'docker exec -it clab-lab4b-PC2 ping -c 3 192.168.100.3', showing successful ping results for 192.168.100.3. The terminal also shows 'ping statistics' for each of these IP addresses, indicating 0% packet loss. The last command shown is 'docker exec -it clab-lab4b-PC1 traceroute 192.168.100.2', which shows a single hop to 192.168.100.2.

```
ivan@ivan-asus:~/pmut/lab4b$ docker exec -it clab-lab4b-PC1 ping -c 3 192.168.100.2
PING 192.168.100.2 (192.168.100.2): 56 data bytes
64 bytes from 192.168.100.2: seq=0 ttl=64 time=1.127 ms
64 bytes from 192.168.100.2: seq=1 ttl=64 time=1.160 ms
64 bytes from 192.168.100.2: seq=2 ttl=64 time=0.994 ms

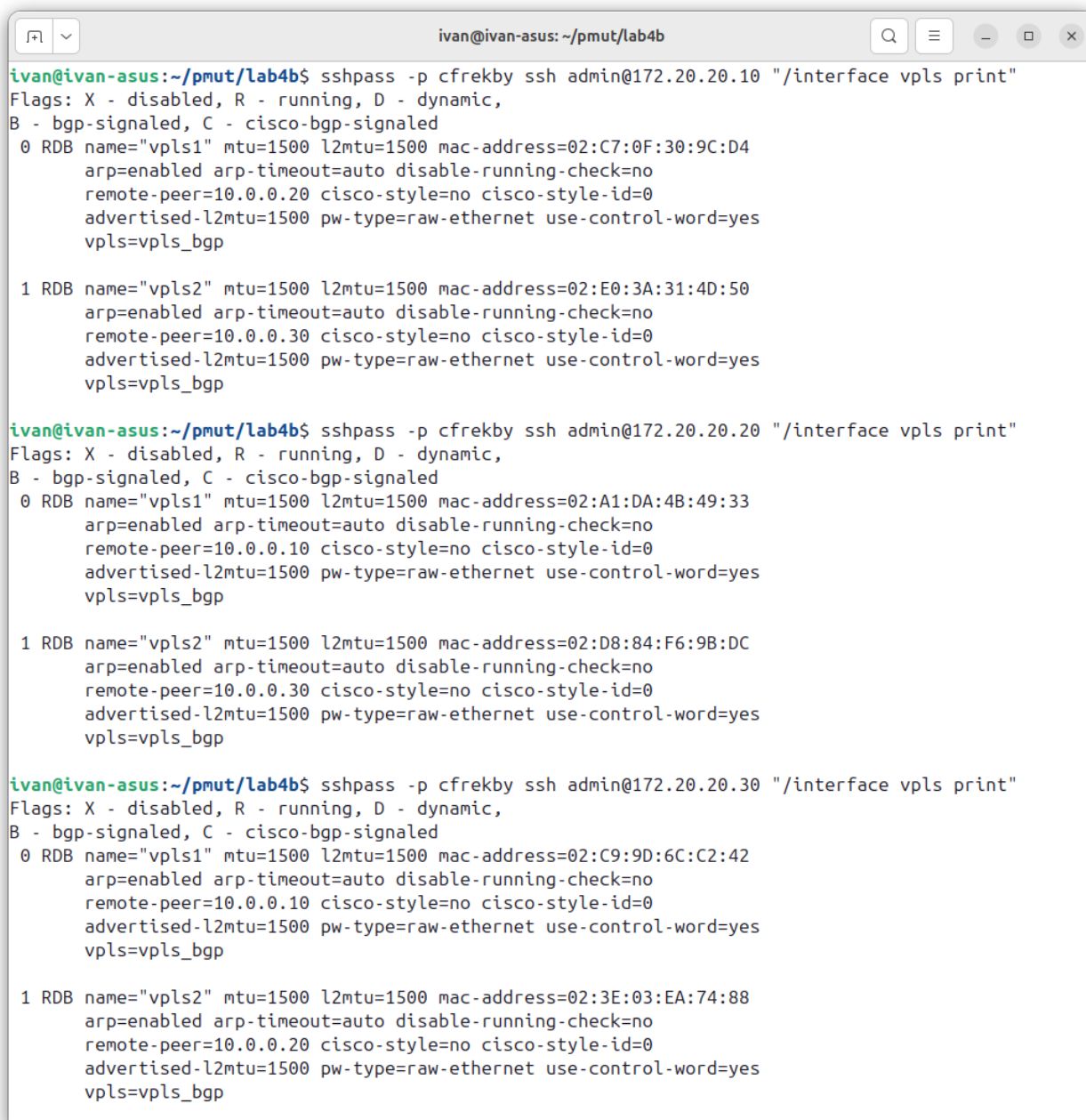
--- 192.168.100.2 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.994/1.093/1.160 ms
ivan@ivan-asus:~/pmut/lab4b$ docker exec -it clab-lab4b-PC1 ping -c 3 192.168.100.3
PING 192.168.100.3 (192.168.100.3): 56 data bytes
64 bytes from 192.168.100.3: seq=0 ttl=64 time=1.120 ms
64 bytes from 192.168.100.3: seq=1 ttl=64 time=0.603 ms
64 bytes from 192.168.100.3: seq=2 ttl=64 time=0.752 ms

--- 192.168.100.3 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.603/0.825/1.120 ms
ivan@ivan-asus:~/pmut/lab4b$ docker exec -it clab-lab4b-PC2 ping -c 3 192.168.100.3
PING 192.168.100.3 (192.168.100.3): 56 data bytes
64 bytes from 192.168.100.3: seq=0 ttl=64 time=1.119 ms
64 bytes from 192.168.100.3: seq=1 ttl=64 time=0.634 ms
64 bytes from 192.168.100.3: seq=2 ttl=64 time=0.849 ms

--- 192.168.100.3 ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.634/0.867/1.119 ms
ivan@ivan-asus:~/pmut/lab4b$ docker exec -it clab-lab4b-PC1 traceroute 192.168.100.2
traceroute to 192.168.100.2 (192.168.100.2), 30 hops max, 46 byte packets
 1 192.168.100.2 (192.168.100.2) 1.161 ms 1.224 ms 0.503 ms
ivan@ivan-asus:~/pmut/lab4b$
```

Рисунок 12 — Проверка связности клиентов в VPLS

Состояние VPLS интерфейсов и изученные MAC-адреса отображены на рисунках 13 и 14.



```
ivan@ivan-asus: ~/pmut/lab4b
ivan@ivan-asus:~/pmut/lab4b$ sshpass -p cfrekby ssh admin@172.20.20.10 "/interface vpls print"
Flags: X - disabled, R - running, D - dynamic,
B - bgp-sigaled, C - cisco-bgp-sigaled
0 RDB name="vpls1" mtu=1500 l2mtu=1500 mac-address=02:C7:0F:30:9C:D4
  arp=enabled arp-timeout=auto disable-running-check=no
  remote-peer=10.0.0.20 cisco-style=no cisco-style-id=0
  advertised-l2mtu=1500 pw-type=raw-ethernet use-control-word=yes
  vpls=vpls_bgp

1 RDB name="vpls2" mtu=1500 l2mtu=1500 mac-address=02:E0:3A:31:4D:50
  arp=enabled arp-timeout=auto disable-running-check=no
  remote-peer=10.0.0.30 cisco-style=no cisco-style-id=0
  advertised-l2mtu=1500 pw-type=raw-ethernet use-control-word=yes
  vpls=vpls_bgp

ivan@ivan-asus:~/pmut/lab4b$ sshpass -p cfrekby ssh admin@172.20.20.20 "/interface vpls print"
Flags: X - disabled, R - running, D - dynamic,
B - bgp-sigaled, C - cisco-bgp-sigaled
0 RDB name="vpls1" mtu=1500 l2mtu=1500 mac-address=02:A1:DA:4B:49:33
  arp=enabled arp-timeout=auto disable-running-check=no
  remote-peer=10.0.0.10 cisco-style=no cisco-style-id=0
  advertised-l2mtu=1500 pw-type=raw-ethernet use-control-word=yes
  vpls=vpls_bgp

1 RDB name="vpls2" mtu=1500 l2mtu=1500 mac-address=02:D8:84:F6:9B:DC
  arp=enabled arp-timeout=auto disable-running-check=no
  remote-peer=10.0.0.30 cisco-style=no cisco-style-id=0
  advertised-l2mtu=1500 pw-type=raw-ethernet use-control-word=yes
  vpls=vpls_bgp

ivan@ivan-asus:~/pmut/lab4b$ sshpass -p cfrekby ssh admin@172.20.20.30 "/interface vpls print"
Flags: X - disabled, R - running, D - dynamic,
B - bgp-sigaled, C - cisco-bgp-sigaled
0 RDB name="vpls1" mtu=1500 l2mtu=1500 mac-address=02:C9:9D:6C:C2:42
  arp=enabled arp-timeout=auto disable-running-check=no
  remote-peer=10.0.0.10 cisco-style=no cisco-style-id=0
  advertised-l2mtu=1500 pw-type=raw-ethernet use-control-word=yes
  vpls=vpls_bgp

1 RDB name="vpls2" mtu=1500 l2mtu=1500 mac-address=02:3E:03:EA:74:88
  arp=enabled arp-timeout=auto disable-running-check=no
  remote-peer=10.0.0.20 cisco-style=no cisco-style-id=0
  advertised-l2mtu=1500 pw-type=raw-ethernet use-control-word=yes
  vpls=vpls_bgp
```

Рисунок 13 — Статус VPLS интерфейсов на PE

```
ivan@ivan-asus: ~/pmut/lab4b
ivan@ivan-asus:~/pmut/lab4b$ sshpass -p cfrekby ssh admin@172.20.20.11 "/mpls ldp neighbor print"
Flags: X - disabled, D - dynamic, O - operational, T - sending-targeted-hello,
V - vpls
#      TRANSPORT      LOCAL-TRANSPORT PEER      SEN
0 DO   10.0.0.33        10.0.0.11        10.0.0.33:0      no
1 DO   10.0.0.22        10.0.0.11        10.0.0.22:0      no
2 DO   10.0.0.10        10.0.0.11        10.0.0.10:0      no

ivan@ivan-asus:~/pmut/lab4b$ sshpass -p cfrekby ssh admin@172.20.20.10 "/interface bridge host print"
Flags: X - disabled, I - invalid, D - dynamic, L - local, E - external
#      MAC-ADDRESS      VID ON-INTERFACE      BRIDGE      AGE
0 DL   9E:F4:EC:E6:AC:89  lo0          lo0          0s
1 DL   02:C7:0F:30:9C:D4  vpls1        br-vpls      0s
2 D    02:C9:9D:6C:C2:42  vpls2        br-vpls      40s
3 D    02:D8:84:F6:9B:DC  vpls2        br-vpls      40s
4 DL   02:E0:3A:31:4D:50  vpls2        br-vpls      0s
5 D    AA:C1:AB:39:9A:74  ether3       br-vpls      26s
6 D    AA:C1:AB:6E:DC:D3  vpls2        br-vpls      40s
7 D    AA:C1:AB:81:25:29  vpls2        br-vpls      26s
8 D    AA:C1:AB:93:00:27  vpls2        br-vpls      37s
9 DL   AA:C1:AB:CB:A1:1A  ether3       br-vpls      0s
10 D   AA:C1:AB:EF:19:1C  vpls2        br-vpls      40s

ivan@ivan-asus:~/pmut/lab4b$
```

Рисунок 14 — Соседи BGP и LDP

Команда `interface bridge host print` на R01.SPB показывает, что MAC-адреса клиентов изучены через VPLS.

ЗАКЛЮЧЕНИЕ

В ходе работы были изучены протоколы OSPF, MPLS, BGP, а также механизмы организации L3VPN (VRF) и L2VPN (VPLS). Собрана IP/MPLS сеть связи с шестью маршрутизаторами, настроены IP-адреса на интерфейсах, OSPF для обеспечения внутренней связности, MPLS LDP для обмена метками, iBGP с использованием route reflectors для передачи VPN-информации. В первой части работы реализован L3VPN с изолированными VRF для трёх клиентских подсетей, что позволило обеспечить связность между ними через MPLS-магистраль. Во второй части настроен VPLS, представляющий эмуляцию Ethernet-сети поверх MPLS, в результате чего все три клиентских устройства оказались в одной широковещательной домене. Проведены проверки связности, проанализированы маршруты и изученные MAC-адреса.

В процессе выполнения работы глобально возник один серьёзный вопрос, связанный с 6 версией ROS. Документация, по которой выполнялись предыдущие и часть этой лабораторной, написана для 7 версии, в которой VRF и VPLS поверх BGP настраивается в них совершенно другими командами, с разными названиями параметров. Разность версий была замечена не сразу, для решения проблемы приходилось искать старые обрывки документации и пробовать команды методом подбора.